PROJECT DOCUMENTATION

School Student Attendance Database Normalization Using SQL

Subject: Database Management Systems (SQL)
Case Study No.: 148 — School Attendance Management
Syllabus Focus: Normalization
Program: B.Tech CSE (2025–29)
University: ITM Skills University
Semester: Semester III

Submitted By: Yuvraj Mishra
Roll Number: 150096725036
Cohort: Jeff Bezos

Submission: Project Report + SQL Script (PostgreSQL)

1. Project Overview

This project redesigns the attendance database of a school using **normalization**. The school currently keeps student names, class information, and daily attendance in one large record structure. Because the same student and class details are written again for every attendance entry, the data is repetitive and difficult to maintain. The database was rebuilt in PostgreSQL (managed through pgAdmin) as three related tables: **students**, **classes**, and **attendance**.

1.1 Problem Statement

A school stores student names, class information, and attendance details in a single large record structure. This causes repeated student and class information. The database must be redesigned using normalization principles.

1.2 Objectives

- Identify the repeating information in the original single-table design.
- Decompose the system into students, classes, and attendance tables.
- Ensure that each table represents exactly one logical entity.
- Create the normalized tables and insert the sample records using SQL.
- Display the attendance of a selected student.
- Calculate the attendance percentage for each student.
- Explain how First, Second, and Third Normal Forms (1NF, 2NF, 3NF) are applied.

1.3 Tools and Technologies

Item	Details
Database System	PostgreSQL (local server)
Client Tool	pgAdmin — Query Tool and ERD Tool
Database Name	school_attendance
Language	SQL (DDL, DML, joins, aggregate functions)
Key Concepts	Primary key, foreign key, UNIQUE constraint, GROUP BY, COUNT, FILTER, ROUND

1.4 Sample Data Used

Students: **Riya** (Class 10-A), **Arjun** (Class 10-A), and **Simran** (Class 10-B). Attendance dates: **2024-07-01** and **2024-07-02**, each with a Present or Absent status. Class teacher names and the specific Present/Absent values are illustrative, since the case study does not fix them.

2. Original Design and Repeating Groups

In the original design, all information is stored in one table, **attendance_raw**. Every row holds the student, the class, the class teacher, the date, and the status together.

```
CREATE TABLE attendance_raw (
    record_id       SERIAL PRIMARY KEY,
    student_name    VARCHAR(50),
    class_name      VARCHAR(10),
    class_teacher   VARCHAR(50),
    attendance_date DATE,
    status          VARCHAR(10)
);

INSERT INTO attendance_raw (student_name, class_name, class_teacher, attendance_date, status) VALUES
('Riya',   '10-A', 'Mrs. Sharma', '2024-07-01', 'Present'),
('Riya',   '10-A', 'Mrs. Sharma', '2024-07-02', 'Present'),
('Arjun',  '10-A', 'Mrs. Sharma', '2024-07-01', 'Present'),
('Arjun',  '10-A', 'Mrs. Sharma', '2024-07-02', 'Absent'),
('Simran', '10-B', 'Mr. Verma',   '2024-07-01', 'Absent'),
('Simran', '10-B', 'Mr. Verma',   '2024-07-02', 'Present');
```

record_id	student_name	class_name	class_teacher	attendance_date	status
1	Riya	10-A	Mrs. Sharma	2024-07-01	Present
2	Riya	10-A	Mrs. Sharma	2024-07-02	Present
3	Arjun	10-A	Mrs. Sharma	2024-07-01	Present
4	Arjun	10-A	Mrs. Sharma	2024-07-02	Absent
5	Simran	10-B	Mr. Verma	2024-07-01	Absent
6	Simran	10-B	Mr. Verma	2024-07-02	Present

Table 2.1 — Contents of the unnormalized attendance_raw table.

2.1 Query: Explaining the Repeating Groups

The query below counts how many times each student–class–teacher combination is stored. Any count above 1 means the same descriptive data has been repeated.

```
SELECT student_name, class_name, class_teacher,
       COUNT(*) AS times_repeated
FROM   attendance_raw
GROUP  BY student_name, class_name, class_teacher
HAVING COUNT(*) > 1;
```

student_name	class_name	class_teacher	times_repeated
Riya	10-A	Mrs. Sharma	2
Arjun	10-A	Mrs. Sharma	2
Simran	10-B	Mr. Verma	2

Figure 2.1 — Each student's class and teacher details are stored twice (once per date).

2.2 Problems Caused by the Repetition

- **Redundancy:** "10-A" and "Mrs. Sharma" appear in four rows; they would appear in every future attendance row for those students.
- **Update anomaly:** if the 10-A class teacher changes, every 10-A row must be edited; missing one leaves inconsistent data.
- **Insertion anomaly:** a new class or student cannot be recorded until at least one attendance entry exists for them.
- **Deletion anomaly:** deleting the only attendance row of a student also erases the student and class information.

3. Normalized Database Design

The single table is decomposed into three tables, each describing one logical entity: a **class**, a **student** who belongs to a class, and an **attendance** record of a student on a date. The ER diagram generated in pgAdmin is shown below; attendance_raw is the original table kept for reference.

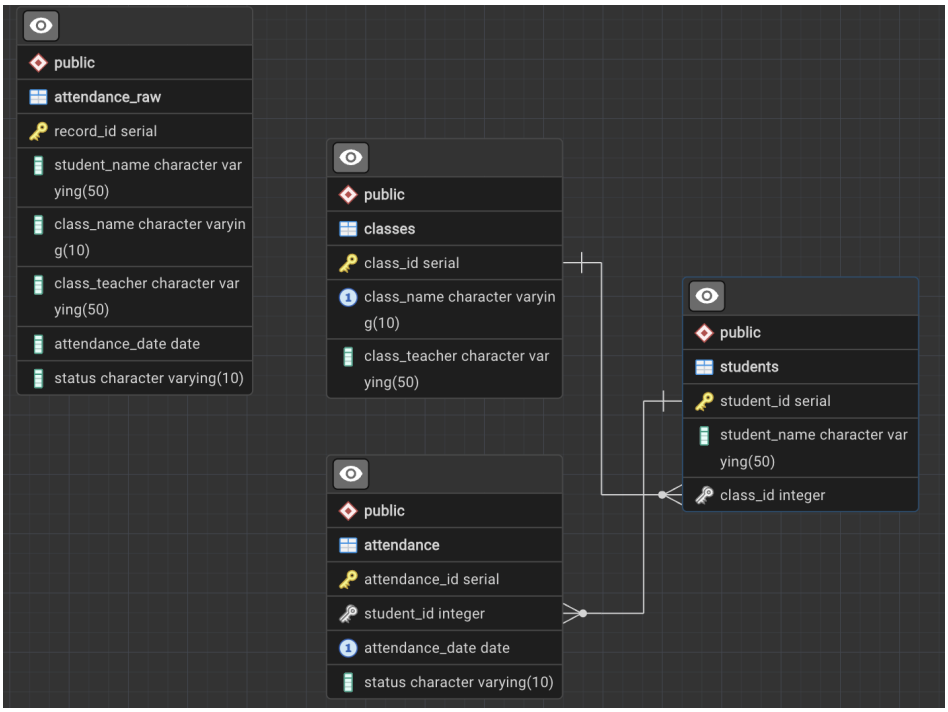


Figure 3.1 — pgAdmin ER diagram of the school_attendance database.

3.1 Table Structure

Table	Column	Type	Key / Constraint
classes	class_id	SERIAL	Primary key
	class_name	VARCHAR(10)	UNIQUE, NOT NULL
	class_teacher	VARCHAR(50)	NOT NULL
students	student_id	SERIAL	Primary key
	student_name	VARCHAR(50)	NOT NULL
	class_id	INTEGER	Foreign key → classes(class_id)
attendance	attendance_id	SERIAL	Primary key
	student_id	INTEGER	Foreign key → students(student_id)
	attendance_date	DATE	UNIQUE with student_id
	status	VARCHAR(10)	CHECK (Present / Absent)

3.2 Relationships

- **classes → students (one-to-many):** one class has many students; each student belongs to one class.
- **students → attendance (one-to-many):** one student has many attendance records; each record belongs to one student.

4. SQL Implementation

4.1 Creating the Normalized Tables

```
CREATE TABLE classes (
    class_id      SERIAL PRIMARY KEY,
    class_name    VARCHAR(10) NOT NULL UNIQUE,
    class_teacher VARCHAR(50) NOT NULL
);

CREATE TABLE students (
    student_id    SERIAL PRIMARY KEY,
    student_name  VARCHAR(50) NOT NULL,
    class_id      INTEGER NOT NULL REFERENCES classes(class_id)
);

CREATE TABLE attendance (
    attendance_id   SERIAL PRIMARY KEY,
    student_id      INTEGER NOT NULL REFERENCES students(student_id),
    attendance_date DATE NOT NULL,
    status          VARCHAR(10) NOT NULL CHECK (status IN ('Present', 'Absent')),
    UNIQUE (student_id, attendance_date)
);
```

4.2 Inserting the Sample Records

The records are moved from attendance_raw into the new tables in dependency order: classes first, then students, then attendance. Using SELECT DISTINCT stores each class and student only once.

```
INSERT INTO classes (class_name, class_teacher)
SELECT DISTINCT class_name, class_teacher
FROM   attendance_raw;

INSERT INTO students (student_name, class_id)
SELECT DISTINCT r.student_name, c.class_id
FROM   attendance_raw r
JOIN   classes c ON c.class_name = r.class_name;

INSERT INTO attendance (student_id, attendance_date, status)
SELECT s.student_id, r.attendance_date, r.status
FROM   attendance_raw r
JOIN   students s ON s.student_name = r.student_name;
```

Result: classes holds 2 rows (10-A, 10-B), students holds 3 rows (Riya, Arjun, Simran), and attendance holds 6 rows (3 students × 2 dates). The class and teacher details are now stored once instead of once per attendance entry.

Table	Rows before (attendance_raw)	Rows after (normalized)
classes	—	2
students	—	3
attendance	6 (all data mixed)	6 (attendance only)

5. Queries and Results

5.1 Display Attendance for a Selected Student

A join across the three tables rebuilds the full picture for one student. The student name in the WHERE clause can be changed to view any other student.

```
SELECT s.student_name, c.class_name, a.attendance_date, a.status
FROM   students s
JOIN   classes c    ON c.class_id   = s.class_id
JOIN   attendance a ON a.student_id = s.student_id
WHERE  s.student_name = 'Riya'
ORDER  BY a.attendance_date;
```

student_name	class_name	attendance_date	status
Riya	10-A	2024-07-01	Present
Riya	10-A	2024-07-02	Present

Table 5.1 — Attendance of Riya (Class 10-A).

5.2 Calculate Attendance Percentage for Each Student

Attendance percentage = (days present ÷ total days recorded) × 100. COUNT(*) FILTER counts only the Present rows, and ROUND limits the result to two decimal places.

```
SELECT s.student_name,
       c.class_name,
       COUNT(*)                                      AS total_days,
       COUNT(*) FILTER (WHERE a.status = 'Present')  AS days_present,
       ROUND(100.0 * COUNT(*) FILTER (WHERE a.status = 'Present')
             / COUNT(*), 2)                          AS attendance_pct
FROM   students s
JOIN   classes c    ON c.class_id   = s.class_id
JOIN   attendance a ON a.student_id = s.student_id
GROUP  BY s.student_id, s.student_name, c.class_name
ORDER  BY s.student_name;
```

student_name	class_name	total_days	days_present	attendance_pct
Arjun	10-A	2	1	50.00
Riya	10-A	2	2	100.00
Simran	10-B	2	1	50.00

Table 5.2 — Attendance percentage for each student.

Arjun is the only student who is absent on 2024-07-02, and Simran is absent on 2024-07-01, so both are at 50.00%. Riya was present on both days (100.00%).

6. How 1NF, 2NF and 3NF Are Applied

Normalization removes redundancy by making every non-key attribute depend only on the key of its own table. In the original table the natural identifier of an attendance fact is the pair (student_name, attendance_date). The functional dependencies in the data are:

- student_name → class_name
- class_name → class_teacher
- (student_name, attendance_date) → status

6.1 First Normal Form (1NF)

Rule: every column holds a single atomic value, there are no repeating groups of columns, and each row is uniquely identified by a primary key.

Application: attendance_raw stores one date and one status per row (not a list like "Present, Absent" in one cell or columns such as day1, day2), and record_id uniquely identifies each row. Every table in the final design also has its own single-column primary key (class_id, student_id, attendance_id) and only atomic values. The design is therefore in 1NF.

6.2 Second Normal Form (2NF)

Rule: the table is in 1NF and no non-key attribute depends on only part of a composite key (no partial dependency).

Problem: using (student_name, attendance_date) as the key, class_name and class_teacher depend only on student_name, which is part of the key. That is a partial dependency, and it is the cause of the repeated student and class data.

Application: student details were moved into the **students** table, and attendance rows keep only the facts that depend on the full key, namely (student_id, attendance_date) → status. Each attendance row now refers to the student through student_id instead of repeating the name and class.

6.3 Third Normal Form (3NF)

Rule: the table is in 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency).

Problem: the chain student → class_name → class_teacher means class_teacher depends on the student only through the class. Teacher details are therefore repeated for every student in the same class.

Application: class_name and class_teacher were moved into the **classes** table. students keeps only class_id as a foreign key. Changing a teacher now needs one update in one row of classes.

6.4 Summary of the Decomposition

Normal Form	Issue in attendance_raw	How it was resolved
1NF	Needs atomic values and a unique key per row	Atomic columns; primary key in every table
2NF	class_name, class_teacher depend only on student_name (partial dependency)	Separate students table; attendance linked by student_id
3NF	class_teacher depends on class_name (transitive dependency)	Separate classes table; students linked by class_id

7. Conclusion

The school attendance database was successfully redesigned from a single repetitive table into three normalized tables: classes, students, and attendance. Each table represents one logical entity and is connected through primary and foreign keys, so student and class information is stored only once. The redesign removes the update, insertion, and deletion anomalies of the original structure, and still allows complete attendance reports to be rebuilt with simple joins.

The SQL queries created the normalized tables, inserted the sample records for Riya, Arjun, and Simran, displayed a selected student's attendance, and calculated each student's attendance percentage. The explanation of 1NF, 2NF, and 3NF shows how each normal form removed a specific kind of redundancy. Overall, the project fulfills the objectives of Case Study 148.